

C++ PROGRAMMING SERIES

EXCEPTION HANDLING IN C++

Master robust error handling · try · catch · throw · Custom Exceptions · Real-World Projects

try / catch / throw

Custom Exceptions

Standard Library

Stack Unwinding

RAII Pattern

Real Projects

TABLE OF CONTENTS

01

Introduction to Exceptions

What, Why, and When to use exceptions

03

Exception Classes & Hierarchy

`std::exception`, built-in types, class hierarchy

05

Stack Unwinding & RAI

Destructors, resource safety, smart pointers

07

Real-World Projects

Bank system, file I/O, web server, calculator

02

try, catch, throw Basics

Syntax, flow of control, multiple catch blocks

04

Custom Exception Classes

Designing your own exception types

06

Re-throwing & Exception Specs

rethrow, noexcept, deprecated specs

08

Best Practices & Anti-Patterns

When to use, performance, guidelines

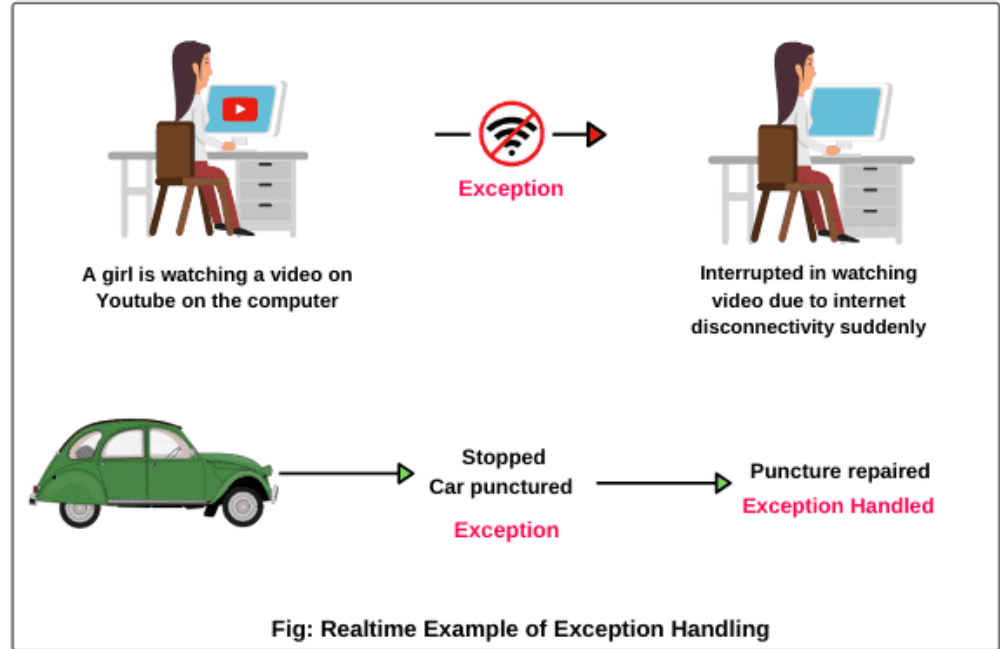
SECTION 01

Introduction to Exceptions

What they are · Why we need them · Real-world motivation

What is an Exception

- Exception is an indication of a problem that occurs during a program's execution
- For example, during execution of the program the control reaches a point where it
 - divides something by zero or
 - reference a memory location that does not exist, etc.



WHAT IS AN EXCEPTION?

An **exception** is an **abnormal condition** that arises during program execution — a situation the normal code is not designed to handle. Exception handling provide a structured mechanism to **separate error-handling code from normal logic**, making programs cleaner, safer, and more maintainable.

✦ What causes them?

- Division by zero
- Null pointer dereference
- File not found
- Array out of bounds
- Invalid user input
- Network failure
- Out of memory

Exception Handling

- Exception handling is a mechanism to **detect and handle runtime errors gracefully**, instead of crashing the program.
- It separates "what the program does" (normal code) from "what happens when it fails" (error-handling code).
- It enables us to write robust and fault-tolerant programs that can deal with problems while **continue executing or terminate the program gracefully**.

WHAT IS AN EXCEPTION?



Without exceptions

- Return codes (-1, NULL)
- Error flags checked everywhere
- Easy to forget error checks
- Mixing logic + error handling
- Deep nesting of if/else
- Prone to silent failures
- Hard to maintain

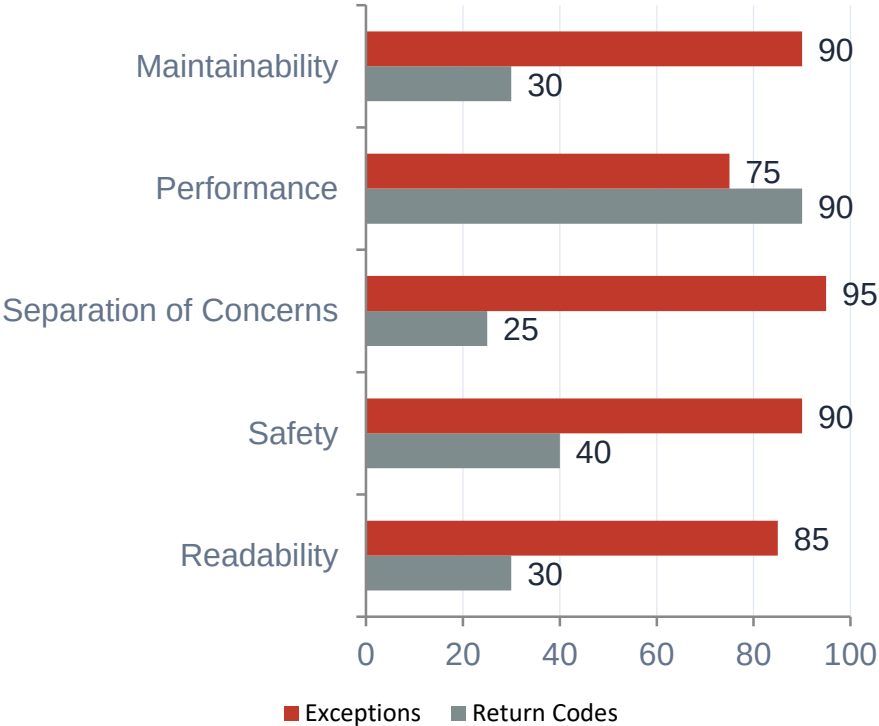


With exceptions

- throw on error, catch elsewhere
- Error handling separated cleanly
- Can't accidentally ignore errors
- Normal code path stays clean
- Propagates automatically
- RAII ensures cleanup
- Readable, maintainable code

EXCEPTIONS vs RETURN CODES — A COMPARISON

Quality Metric Comparison (0–100)



	Return Codes	Exceptions
Propagation	Manual (must check each call)	Automatic — bubbles up
Forgettable?	Yes — silent failures	No — unhandled = terminate
Code clarity	Cluttered with checks	Clean normal path
Object info	Just an int/enum	Full exception object
With RAII	Manual cleanup needed	Automatic resource cleanup
Best for	Local, simple errors	Cross-function error handling

SECTION 02

try - catch - throw

Basics

Syntax · Control flow · Multiple catch blocks · Catch-all

How to handle an Exception

- Piece of program that throws an exception
- If an exception handler exists, it deals with the error ➡ It catches and handles exception
- If no exception handler exists, uncaught exception ➡ Could terminate program
- **try** block encloses code that may raise exception
- One or more **catch** blocks to
 - Catch and handle exception, if appropriate

try — block of code that might throw an error
throw — signals that an error has occurred
catch — handles the error

```
try {  
    // code that might fail  
    throw someValue; // if error occurs  
}  
catch (type variable) {  
    // handle the error  
}
```

```
try {  
    // code that may raise exception code  
}  
catch( ExceptionName e1 ){  
    // catch block  
}  
catch( ExceptionName e2 ) {  
    // catch block  
}  
catch( ExceptionName eN ) {  
    // catch block  
}
```

THE try / catch / throw MECHANISM

SYNTAX TEMPLATE

```
// Basic syntax
try {
    // Code that might throw
    int result = divide(10, 0);    // throws if divisor == 0
    cout << "Result: " << result; // skipped if exception thrown
}
catch (const std::invalid_argument& e) {
    // Handle specific exception type
    cout << "Error: " << e.what();
}
catch (const std::exception& e) {
    // Catch any std exception (broader net)
    cout << "Std exception: " << e.what();
}
catch (...) {
    // Catch AaNYTHING (last resort)
    cout << "Unknown exception caught!";
}
// Execution continues here after catch block
```

try block enters

Exception thrown?

Yes

NO: skip catch

Match catch type

Handler executes

Continue after catch

MULTIPLE CATCH BLOCKS & CATCH-ALL

MULTIPLE CATCH – C++ CODE

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

double safeDivide(double a, double b) {
    if (b == 0.0)
        throw invalid_argument("Division by zero is undefined");
    return a / b;
}

int getAge(int age) {
    if (age < 0 || age > 150)
        throw out_of_range("Age " + to_string(age) + " is not valid");
    return age;
}

int main() {
    try {
        cout << safeDivide(10.0, 0.0) << endl; // throws invalid_argument
        cout << getAge(-5) << endl; // never reached
        catch (const invalid_argument& e) { // most specific first!
            cout << "Invalid argument: " << e.what() << endl;
        }
        catch (const out_of_range& e) {
            cout << "Out of range: " << e.what() << endl;
        }
        catch (const exception& e) { // catches any std::exception
            cout << "Standard error: " << e.what() << endl;
        }
        catch (...) { // catches ANYTHING including non-std
            cout << "Unknown error occurred!" << endl;
        }

        cout << "Program continues normally." << endl;
        return 0;
    }
}
```

Key Rules

1

Most specific FIRST

Derived exception types must appear before base types, or they'll never be reached

2

catch(...) is last resort

Catches everything including non-C++ exceptions. Put it last, always

3

Catch by const ref

Use const& to avoid slicing and unnecessary copying of exception objects

4

Matched in order

C++ tests each catch in sequence — first match wins, rest are skipped

5

No match → propagate

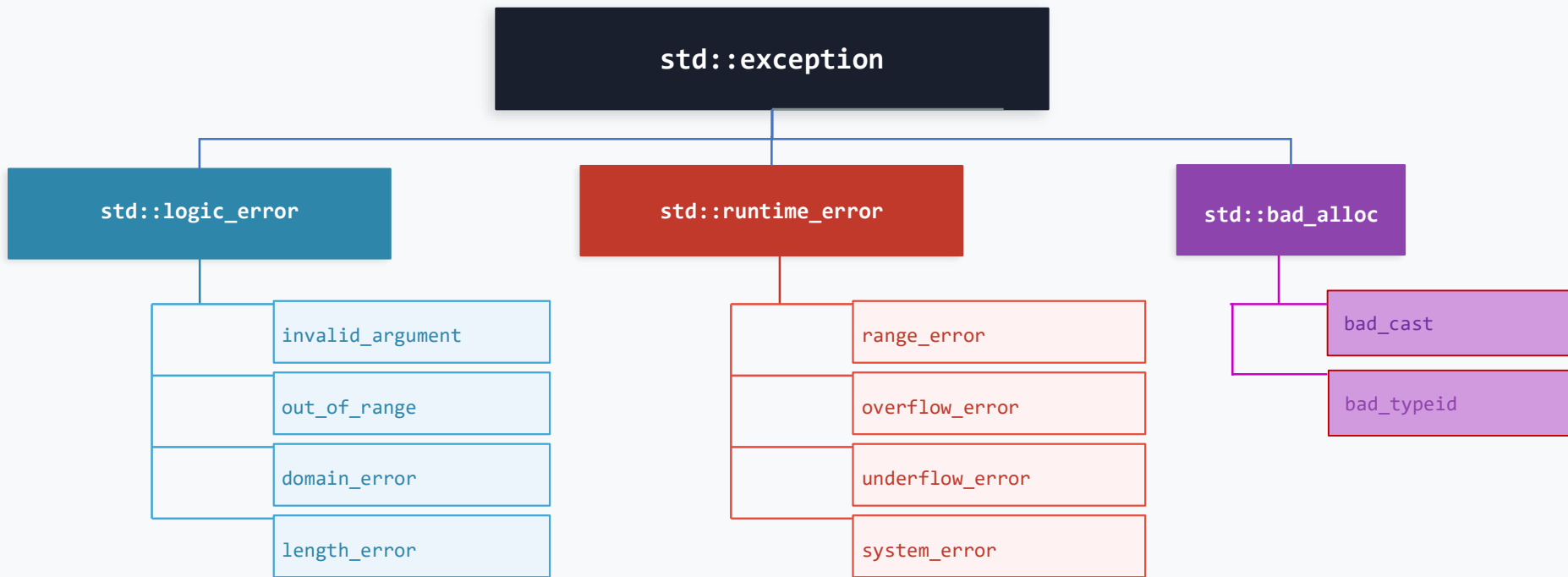
If no catch matches, exception propagates up the call stack further

SECTION 03

Exception Classes & Hierarchy

std::exception · built-in types · class hierarchy tree

C++ STANDARD EXCEPTION HIERARCHY



All inherit from `std::exception` and override `what()` which returns a `const char*` description of the error.

STANDARD EXCEPTION TYPES — WHEN TO USE EACH

`std::invalid_argument`

`<stdexcept>`

Function receives argument that violates precondition

```
void setAge(int a) {  
    if(a<0||a>150)  
        throw invalid_argument(  
            "Age must be 0-150");  
}
```

`std::out_of_range`

`<stdexcept>`

Index or value outside valid range (e.g., `vector::at()`)

```
string s = "hello";  
char c = s.at(10);  
// throws out_of_range  
// if index >= s.size()
```

`std::runtime_error`

`<stdexcept>`

Errors detectable only at runtime (network, file, format)

```
if(!file.is_open())  
    throw runtime_error(  
        "Cannot open config.txt");  
// Detected only at runtime
```

`std::overflow_error`

`<stdexcept>`

Arithmetic overflow: result too large for the type

```
if(a>INT_MAX-b)  
    throw overflow_error(  
        "Integer overflow  
detected");
```

`std::bad_alloc`

`<new>`

new operator fails to allocate memory

```
try {  
    int* p = new int[1e15];  
}  
catch(const bad_alloc& e){  
    // Out of memory!  
}
```

`std::logic_error`

`<stdexcept>`

Errors in program logic that could be caught at compile time

```
// Base class for  
// invalid_argument,  
// out_of_range,  
// domain_error, etc.
```

SECTION 04

Custom Exception Classes

Designing your own · Inheritance · what() override · Rich error data

DESIGNING CUSTOM EXCEPTION CLASSES

CUSTOM EXCEPTION CLASSES – BANK EXAMPLE

```
#include <stdexcept>
#include <string>
using namespace std;

// —— Custom base exception ——
class BankException : public runtime_error
{ int errorCode_;
public:
    BankException(const string& msg, int
                  code = 0) : runtime_error(msg),
                             errorCode_(code) {}

    int errorCode() const noexcept {
        return errorCode_ ; }
};

// —— Specialized exceptions ——
class InsufficientFundsException : public
BankException {
    double balance_, amount_;
public:
    InsufficientFundsException(double bal,
double amt):BankException("Insufficient
funds: need $" + to_string(amt-bal) + "
more", 1001), balance_(bal), amount_(amt){}

    double shortfall() const {
        return amount_ - balance_ ; }
};

class AccountLockedException : public
BankException {
    string accountId_;
public:
    AccountLockedException(const string& id)
        :BankException("Account " + id + " is
        locked", 1002), accountId_(id) {}

    string accountId() const {
        return accountId_ ; }
};

// —— Using the custom exceptions ——
class BankAccount {
    double balance_ = 500.0;
    bool locked_ = false;
    string id_ = "ACC-001";
public:
    void withdraw(double amount) {
        if (locked_)
            throw AccountLockedException(id_);
        if (amount > balance_)
            throw InsufficientFundsException(
                balance_ , amount);

        balance_ -= amount;
    }
};
```

Design Guidelines

- ✓ Inherit from `std::exception` or its subclasses
- ✓ Prefer `runtime_error` for runtime issues, `logic_error` for logic bugs
- ✓ Always provide a meaningful `what()` message
- ✓ Add extra fields for rich error context (error code, amounts, IDs)
- ✓ Mark `what()` as `noexcept` – it must not throw
- ✓ Use an exception hierarchy for your domain (`BankException` → `InsufficientFunds`)
- ⚠ Don't throw exceptions from destructors – causes `std::terminate`
- ⚠ Don't put heavy logic in exception constructors

REAL PROJECT: ATM BANKING SYSTEM

ATM EXCEPTION HANDLING – FULL EXAMPLE

```
void BankAccount::withdraw(double amount)
{
    if (locked_)
        throw AccountLockedException(id_);
    if (amount > balance_)
        throw InsufficientFundsException(
            balance_, amount);

    balance_ -= amount;
    cout << "Withdrew $" << amount << ".
        Balance: $" << balance_ << endl;
}

int main() {
    BankAccount acc;
    try {
        acc.withdraw(200.0);    // ✓ OK
        acc.withdraw(400.0);    // ✗
        throws InsufficientFundsException
    }
    catch(const InsufficientFundsException& e) {
        // Rich error data
        cout << "ERROR [" << e.errorCode()
            << "]: " << e.what() << endl;
        cout << "Shortfall: $" << e.shortfall()
            << endl;
    }
}

catch (const AccountLockedException& e) {
    cout << "LOCKED [" << e.errorCode()
        << "]: " << e.what() << endl;
    cout << "Account ID: " << e.accountId()
        << endl;
}

catch (const BankException& e) {
    // Catches any bank error
    cout << "Bank error [" << e.errorCode()
        << "]: " << e.what() << endl;
}

catch (const exception& e) {
    cout << "Unexpected: " << e.what()
        << endl;
}

return 0;
}
```

PROGRAM OUTPUT

```
Withdrew $200.00. Balance: $300.00
ERROR [1001]: Insufficient funds:
    need $100.00 more
Shortfall: $100.00
```

Exception Hierarchy

std::exception

std::runtime_error

BankException

InsufficientFunds

AccountLocked

SECTION 05

Stack Unwinding & RAII

How exceptions clean up · Destructors · Resource safety

STACK UNWINDING — HOW C++ CLEANS UP

Stack Unwinding: When an exception is thrown, C++ automatically destroys all local objects in the current scope (and each calling scope) in the reverse order of their construction — working up the call stack until a matching catch block is found.

CALL STACK (bottom = most recent)

main()
catch block handles exception

readConfig()
file opened → exception thrown → file closed

parseINI()
vector created → destroyed during unwind

processLine()
throws runtime_error("bad format")

↑ unwinds up

← throw!

RAII + STACK UNWINDING DEMO

```
#include <fstream>
#include <stdexcept>
using namespace std;

class FileGuard {
    fstream file_;
public:
    FileGuard(const string& name)
        : file_(name) {
        if (!file_.is_open())
            throw runtime_error("Cannot
                                open: " + name);
        cout << "File opened";
    }
    ~FileGuard() {
        file_.close();
        // ALWAYS runs!
        cout << "File closed";
    }
};

void parseINI() {
    FileGuard fg("config.ini");
    processLine(); // throws!
    // fg.~FileGuard() called auto
}

int main() {
    try { parseINI(); }
    catch(const exception& e) {
        cout << e.what();
    }
}

/* Output:
File opened
File closed    <-- RAII works!
Bad format
*/

void processLine() {
    throw runtime_error("Bad
                        format");
}
```

RAII — RESOURCE ACQUISITION IS INITIALIZATION

RAII: Tie resource lifetime to object lifetime. Constructor acquires the resource. Destructor releases it. Since destructors run during stack unwinding, resources are **always freed** — even when exceptions are thrown.

✗ WITHOUT RAI (Dangerous)

```
void processData() {
    int* data = new int[1000];
    FILE* f = fopen("data.txt", "r");

    parseData(data); // THROWS!

    // These NEVER execute:
    delete[] data;    // MEMORY LEAK!
    fclose(f);        // FILE LEAK!
}
// With return codes, you must
// remember every cleanup path.
// Easy to forget. Bug-prone.
```

✓ WITH RAI (Exception-Safe)

```
#include <memory>
#include <fstream>
void processData() {
    // RAI wrappers
    // smart pointer — automatic management - automatically
    // deleted
    auto data = make_unique<int[]>(1000);
    // same as: unique_ptr<int[]> data = make_unique<int[]>(1000);
    fstream f("data.txt");

    parseData(data.get()); // THROWS
    // Destructors auto-run:
    // data freed ✓
    // f closed ✓
}
// No manual cleanup needed.
// Exception-safe by design!
// std::unique_ptr, shared_ptr,
// fstream are all RAI classes.
```

unique_ptr is a smart pointer — it wraps a raw pointer and automatically calls `delete[]` when it goes out of scope:

SECTION 06

Re-throwing & noexcept

Rethrow · noexcept specifier · exception_ptr · nested_exception

RE-THROWING · noexcept · MODERN EXCEPTION SPECS

ADVANCED EXCEPTION TECHNIQUES

```
// — 1. Re-throw with bare throw; —  
void logAndRethrow() {  
    try {  
        riskyOperation();  
    }  
    catch (const exception& e) {  
        cerr << "[LOG] Exception: " << e.what()  
            << endl;  
        throw; // Re-throws the ORIGINAL  
              // exception (keeps type)  
        // Never use: throw e; (that SLICES the  
        // object!)  
    }  
}
```

```
// — 2. noexcept — promise that function  
// won't throw —  
int safeAdd(int a, int b) noexcept {  
    // Guaranteed no-throw  
    return a + b;  
}  
// If it does throw, std::terminate() is  
// called which immediately kills the program — no  
// cleanup, no catch blocks, no second chances. It is  
// the last resort when C++ considers the situation  
// unrecoverable.
```

```
// — 3. noexcept(expr) — conditional  
noexcept —  
template<typename T>  
void swap_safe(T& a, T& b)  
noexcept(noexcept(T(std::move(a)))) {  
    T temp = std::move(a);  
    a = std::move(b);  
    b = std::move(temp);  
}
```

```
// — 4. std::exception_ptr — store &  
// transport exceptions —  
std::exception_ptr g_ex = nullptr;
```

```
void workerThread() {  
    try { riskyWork(); }  
    catch (...) {  
        g_ex = std::current_exception(); } //Capture  
}
```

```
void checkException() {  
    if (g_ex)  
        // Re-throw in another thread  
        std::rethrow_exception(g_ex);  
}
```

Quick Reference

throw;

Re-throws current exception unchanged (preserves type/data) - rethrow exact same exception — don't create a new one.

throw e;

Creates a COPY → may slice derived to base! Avoid.

noexcept

Declares function will never propagate exceptions

noexcept(expr)

Conditionally noexcept based on parameter (true/false)

current_exception()

Captures the currently active exception and stores it in an exception_ptr so you can handle it later

rethrow_exception(p)

Re-throws a stored exception_ptr anywhere

SECTION 07

Real-World Projects

Calculator • File processor • Web server • Student grade system

PROJECT 1: SAFE SCIENTIFIC CALCULATOR

SAFE CALCULATOR WITH CUSTOM EXCEPTIONS

```
#include <iostream>
#include <stdexcept>
#include <cmath>
using namespace std;

// Custom exceptions for calculator
class CalcException : public runtime_error {
public: CalcException(const string& m) : runtime_error(m) {}
};
class DivByZeroException : public CalcException {
public: DivByZeroException() : CalcException("Division by zero") {}
};
class NegativeSqrtException: public CalcException {
public: NegativeSqrtException(double v)
      : CalcException("Square root of negative: " + to_string(v)) {}
};
class LogNonPositiveException: public CalcException {
public: LogNonPositiveException(double v)
      : CalcException("Log of non-positive: " + to_string(v)) {}
};

class SafeCalculator {
public:
    static double divide(double a, double b) {
        if (b == 0) throw DivByZeroException();
        return a / b;
    }
    static double squareRoot(double x) {
        if (x < 0) throw NegativeSqrtException(x);
        return sqrt(x);
    }
    static double logarithm(double x) {
        if (x <= 0) throw LogNonPositiveException(x);
        return log(x);
    }
    static double power(double base, int exp) noexcept {
        return pow(base, exp);    // Always safe
    }
};

int main() {
    auto test = [](const string& op, auto fn) {
        try { cout << op << " = " << fn() << endl; }
        catch (const CalcException& e) { cout << "[Calc] " << e.what()
        << endl; }
        catch (const exception& e) { cout << "[Err] " << e.what()
        << endl; }
    };
    test("10/2", []{ return SafeCalculator::divide(10, 2); });
    test("10/0", []{ return SafeCalculator::divide(10, 0); }); //
    throws
    test("sqrt(9)",[]{ return SafeCalculator::squareRoot(9); });
    test("sqrt(-4)",[]{ return SafeCalculator::squareRoot(-4); }); //
    throws
    test("log(0)", []{ return SafeCalculator::logarithm(0); }); //
    throws
    test("2^10", []{ return SafeCalculator::power(2, 10); });
}
```


PROJECT 2: STUDENT GRADE MANAGEMENT SYSTEM

STUDENT GRADE SYSTEM – EXCEPTION-SAFE DESIGN

```
#include <iostream>
#include <vector>
#include <string>
#include <stdexcept>
using namespace std;

// — Domain exceptions —
class StudentException : public runtime_error {
    string studentId_;
public:
    StudentException(const string& msg, const string& id)
        : runtime_error(msg), studentId_(id) {}
    string studentId() const { return studentId_; }
};
class InvalidGradeException: public StudentException {
    double grade_;
public:
    InvalidGradeException(double g, const string& id)
        : StudentException("Grade " + to_string(g) +
            " out of [0,100] for " + id, id), grade_(g) {}
    double grade() const { return grade_; }
};
class StudentNotFoundException : public StudentException {
public:
    StudentNotFoundException(const string& id)
        : StudentException("Student not found: " + id, id) {}
};

// — Student record —
struct Student {
    string id, name;
    vector<double> grades;

    void addGrade(double g) {
        if (g < 0 || g > 100)
            throw InvalidGradeException(g, id);
        grades.push_back(g);
    }
    double average() const {
        if (grades.empty())
            throw StudentException("No grades for " + id, id);
        double sum = 0;
        for (double g : grades) sum += g;
        return sum / grades.size();
    }
};

// — Grade book —
class GradeBook {
```

```
    vector<Student> students_;
public:
    void addStudent(Student s) { students_.push_back(s); }

    Student& findStudent(const string& id) {
        for (auto& s : students_)
            if (s.id == id) return s;
        throw StudentNotFoundException(id);
    }
    void printReport() {
        for (auto& s : students_) {
            try {
                cout << s.name << " (" << s.id << "): avg = "
                    << s.average() << endl;
            } catch (const StudentException& e) {
                cout << "[WARN] " << e.what() << endl; // partial failure OK
            }
        }
    }
};

int main() {
    GradeBook gb;
    gb.addStudent({"S001", "Ali", {}});
    gb.addStudent({"S002", "Sara", {}});

    try {
        gb.findStudent("S001").addGrade(85);
        gb.findStudent("S001").addGrade(92);
        gb.findStudent("S001").addGrade(110); // throws InvalidGradeException
    }
    catch (const InvalidGradeException& e) {
        cerr << "Invalid grade " << e.grade() << " for " << e.studentId() << endl;
    }
    try { gb.findStudent("S999"); } // throws StudentNotFoundException
    catch (const StudentNotFoundException& e) {
        cerr << "Lookup failed: " << e.what() << endl;
    }
    gb.printReport();
}
```

PROJECT 3: ROBUST FILE PROCESSOR

FILE PROCESSOR – LAYERED EXCEPTION HANDLING

```
#include <fstream>
#include <sstream>
#include <vector>
#include <stdexcept>
using namespace std;

class FileException : public runtime_error {
    string filename_;
public:
    FileException(const string& msg, const string& fn)
        : runtime_error(msg + ": " + fn), filename_(fn) {}
    string filename() const { return filename_; }
};

class ParseException : public runtime_error {
    int lineNum_;
public:
    ParseException(const string& msg, int line)
        : runtime_error("Line " + to_string(line) + ": " + msg), lineNum_(line) {}
    int lineNumber() const { return lineNum_; }
};

struct Record { string name; int age; double salary; };

vector<Record> loadCSV(const string& filename) {
    ifstream file(filename);
    if (!file.is_open())
        throw FileException("Cannot open file", filename);

    vector<Record> records;
    string line;
    int lineNum = 0;

    while (getline(file, line)) {
        ++lineNum;
        if (line.empty() || line[0] == '#') continue; // skip comments

        istringstream ss(line);
        Record r;
        string ageStr, salStr;

        if (!getline(ss, r.name, ',') ||
            !getline(ss, ageStr, ',') ||
            !getline(ss, salStr, ','))
            throw ParseException("Expected 3 columns", lineNum);

        try {
            r.age = stoi(ageStr);
            r.salary = stod(salStr);
        } catch (const invalid_argument&) {
            throw ParseException("Non-numeric age or salary", lineNum);
        } catch (const out_of_range&) {
            throw ParseException("Value too large", lineNum);
        }

        if (r.age < 18 || r.age > 100)
            throw ParseException("Age " + ageStr + " out of [18,100]", lineNum);
        if (r.salary < 0)
            throw ParseException("Negative salary", lineNum);

        records.push_back(r);
    }
    return records;
}

int main() {
    try {
        auto records = loadCSV("employees.csv");
        cout << "Loaded " << records.size() << " records."
        << endl;

        for (auto& r : records)
            cout << r.name << ", age " << r.age << ", $" << r.salary << "
            << endl;

    } catch (const FileException& e) { cerr << "[FILE] " << e.what() << endl; }
    catch (const ParseException& e) { cerr << "[PARSE] " << e.what() << endl; }
    catch (const exception& e) { cerr << "[ERROR] " << e.what() << endl; }
}
```

PROJECT 4: WEB SERVER REQUEST HANDLER

WEB SERVER — HTTP EXCEPTION HIERARCHY

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

// — HTTP-style exceptions —
class HttpException : public runtime_error {
    int statusCode_;
public:
    HttpException(int code, const string& msg)
        : runtime_error(msg), statusCode_(code) {}
    int statusCode() const { return statusCode_; }
};

class NotFoundException : public HttpException {
public:
    NotFoundException(const string& res)
        : HttpException(404, "Not Found: " + res) {}
};

class UnauthorizedException : public HttpException {
public:
    UnauthorizedException()
        : HttpException(401, "Unauthorized: missing or invalid token") {}
};

class BadRequestException : public HttpException {
public:
    BadRequestException(const string& reason)
        : HttpException(400, "Bad Request: " + reason) {}
};

class InternalServerErrorException : public HttpException {
public:
    InternalServerErrorException(const string& detail)
        : HttpException(500, "Internal Server Error: " + detail) {}
};

// — Request processing —
struct Request { string method, path, token, body; };
struct Response { int status; string body; };

Response handleRequest(const Request& req) {
    if (req.token.empty())
        throw UnauthorizedException();
    if (req.method != "GET" && req.method != "POST")
        throw BadRequestException("Unsupported method: " + req.method);
    if (req.path == "/api/users")
```

```
        return {200, R"({"users":["ali","sara"]})"};
    if (req.path == "/api/admin")
        throw NotFoundException("/api/admin");
    throw NotFoundException(req.path);
}

// — Middleware-style exception handler —
Response safeHandle(const Request& req) noexcept {
    try {
        return handleRequest(req);
    }
    catch (const HttpException& e) {
        return {e.statusCode(), string("Error: ") + e.what()};
    }
    catch (const exception& e) {
        return {500, string("Internal: ") + e.what()};
    }
    catch (...) {
        return {500, "Unknown server error"};
    }
}

int main() {
    vector<Request> requests = {
        {"GET", "/api/users", "token123", ""}, // 200 OK
        {"GET", "/api/users", "", ""}, // 401
        {"GET", "/api/admin", "token123", ""}, // 404
        {"POST", "/api/users", "token123", "{}"}, // 200 OK
        {"PUT", "/api/data", "token123", ""}, // 400 Bad Method
    };
    for (auto& req : requests) {
        auto res = safeHandle(req);
        cout << req.method << " " << req.path
              << " -> " << res.status << " " << res.body << "
    };
}
```

SECTION 08

Best Practices & Anti-Patterns

When to use · Performance · Guidelines · Common mistakes

BEST PRACTICES FOR EXCEPTION HANDLING

✓ Catch by const reference

`catch(const exception& e)` — avoids slicing and unnecessary copies. Never catch by value.

✓ Most specific catch first

Order matters: catch derived types before base types, else they're unreachable.

✓ Inherit from `std::exception`

Ensures compatibility with `catch(exception&)` and uniform `what()` interface.

⚠ Exceptions for EXCEPTIONAL events

Not for flow control or routine cases. Performance penalty — exceptions are slow when thrown.

✗ Don't throw from destructors

If destructor throws during stack unwinding, `std::terminate` is called. Fatal.

✗ Never catch and ignore (empty catch)

`catch(exception& e) {}` silently hides bugs. At minimum, log.

✓ Throw by value

`throw MyException(...)` — not `throw new MyException(...)`. Stack-allocated, no memory management.

✓ Use `noexcept` where appropriate

Mark functions that truly can't fail. Enables compiler optimizations and move semantics.

✓ RAI for all resources

Wrap files, memory, locks, sockets in RAI classes or use `unique_ptr`/`shared_ptr`.

⚠ Document which exceptions are thrown

Add `@throws` to comments. Consider `noexcept` for guaranteed no-throw functions.

✗ Don't use exceptions for every error

Return codes or `std::optional` suit local, performance-critical, or expected errors.

✗ Don't throw new — use stack objects

`throw new X()` creates leak risk. Use `throw X()` instead.

EXCEPTION HANDLING PERFORMANCE & WHEN TO USE

USE EXCEPTIONS WHEN:

- ✓ Error cannot be handled locally
- ✓ Error must propagate up many levels
- ✓ Constructors fail (no return value!)
- ✓ Recoverable at a higher level
- ✓ Contract/precondition violated
- ✓ Network / file / IO failures

AVOID EXCEPTIONS WHEN:

- ✗ Expected/common conditions
- ✗ Performance-critical paths
- ✗ Real-time systems with tight timing
- ✗ Simple local error (use optional<>)

Key insight: Zero overhead when NO exception is thrown (zero-cost exception model). Exceptions only cost when actually thrown — typically 100-1000× slower than normal returns. Ideal for RARE error paths, not hot loops.

COMMON ANTI-PATTERNS & HOW TO FIX THEM

✗ Catch by value + empty catch

```
catch (exception e) { }
```

✓ Catch by const ref + log always

```
catch (const exception& e) {  
    cerr << e.what();  
}
```

✗ Massive try block + swallow all

```
try {  
    doA(); doB(); doC();  
    doD(); doE();  
} catch (...) { }
```

✓ Fine-grained, specific handling

```
// Fine-grained try blocks  
try { doA(); } catch(const X& e){handleX(e);}  
try { doB(); } catch(const Y& e){handleY(e);}
```

✗ Manual memory without RAI

```
int* p = new int[100];  
doWork(); // might throw  
delete[] p; // NEVER REACHED
```

✓ RAI with unique_ptr

```
auto p = make_unique<int[]>(100);  
doWork(); // throws? p auto-freed
```

✗ throw new (heap allocation)

```
throw new MyException("err");  
// Caller must delete! Leak-prone
```

✓ throw by value (stack object)

```
throw MyException("err");  
// Stack object, auto-managed
```

THREE EXCEPTION SAFETY GUARANTEES



No-throw (Strongest)

Function ALWAYS succeeds. Will NEVER throw or propagate an exception. Resources always cleaned up.

How to achieve:

Mark with `noexcept`. Use only operations that cannot fail.

Examples:

- Destructors
- `swap()` functions
- Move constructors
- Simple value returns



Strong (Commit/Rollback)

Operation either succeeds completely, OR fails with program state exactly as before the call. No partial changes.

How to achieve:

Use copy-and-swap idiom. Make changes to temp copy, then swap at the end (swap is `noexcept`).

Examples:

- `vector::push_back`
- File write with atomic rename
- DB transactions
- Copy-assignment



Basic (Minimum)

If exception is thrown: no resources are leaked and object invariants are maintained. State may have changed.

How to achieve:

Use RAII for all resources. Don't leak memory or handles. Don't break class invariants.

Examples:

- Most STL operations
- File append
- Multi-step algorithms
- Network operations

QUICK REFERENCE CHEAT SHEET

SYNTAX REFERENCE

```
// — Basic syntax
try {
    riskyFunction();
}
catch (const SomeException& e) {
    cerr << e.what();
}
catch (...) { /* catch-all */ }

// — Throw
throw invalid_argument("bad value");

// — Re-throw (preserve type!)
catch (const exception& e) {
    log(e.what());
    throw; // NOT throw e;
}

// — noexcept
void safeSwap(T& a, T& b)
noexcept;

// — Custom exception
class MyEx : public
runtime_error {
public:
    MyEx(const string& m, int
code)
        : runtime_error(m),
code_(code) {}
    int code() const { return
code_; }
private:
    int code_;
};

// — RAII (auto resource
cleanup)
{
    unique_ptr<int[]> p(new
int[100]);
    fstream f("data.txt");
    riskyWork();
} // p and f auto-destroyed here
```

Standard Exceptions at a Glance

<code>invalid_argument</code>	Bad function argument value
<code>out_of_range</code>	Index/value outside valid range
<code>runtime_error</code>	Runtime-only detectable error
<code>overflow_error</code>	Arithmetic overflow
<code>underflow_error</code>	Arithmetic underflow
<code>bad_alloc</code>	new/new[] fails (out of memory)
<code>bad_cast</code>	Failed dynamic_cast to reference
<code>bad_typeid</code>	typeid on null pointer
<code>domain_error</code>	Math domain violation
<code>length_error</code>	Max size exceeded (e.g. string)
<code>logic_error</code>	Base for logic exception group

All in <stdexcept> | `bad_alloc` in <new> | `bad_cast`/`typeid` in <typeinfo>

EXCEPTION HANDLING IN C++

THANK YOU